

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação

SSC0903 — Computação de Alto Desempenho
Primeiro Trabalho Prático

Incêndio Florestal

Implementações sequencial e paralela com OpenMP

Integrante	Número USP
João Ricardo de Almeida Lustosa	15453697
Catarina Moreira Lima	8957221
Nome completo	Número USP

São Carlos
Setembro de 2026

Sumário

1	Introdução	3
2	Objetivos	3
2.1	Objetivo geral	3
2.2	Objetivos específicos	3
3	Modelagem do problema	4
3.1	Domínio espacial e temporal	4
3.2	Parâmetros e restrições da entrada	4
3.3	Cobertura, umidade e inicialização determinística	4
3.4	Estados e estruturas de armazenamento	5
3.5	Zonas de contenção	5
3.6	Vizinhança e potencial de ignição	6
3.7	Transições, atualização síncrona e parada	6
3.8	Estatísticas e saída	7
4	Implementação sequencial	7
4.1	Organização do código	7
4.2	Preparação da simulação	7
4.3	Laço principal	8
4.4	Finalização e custo	9
5	Implementação paralela	9
5.1	Estratégia de paralelização	9
5.2	Compartilhamento e sincronização	10
5.3	Escalonamento	11
6	Validação da correção	11
7	Ambiente e metodologia experimental	12
7.1	Ambiente das duas versões	12
7.2	Cargas e protocolo de execução	13
7.3	Delimitação do trecho cronometrado	13
7.4	Métricas	13
8	Resultados	13
8.1	MacOs	13
9	Análise e discussão	14

1 Introdução

Simulações em grades discretas permitem representar a evolução de um sistema por meio de regras locais aplicadas repetidamente a suas células. Neste trabalho, uma matriz retangular representa uma área florestal na qual o fogo se propaga a partir de focos iniciais. A evolução depende da cobertura e da umidade de cada célula, da presença de vizinhos em chamas, da direção e da intensidade do vento e da ativação de zonas de contenção em instantes predeterminados [1].

O modelo utiliza passos discretos e atualização síncrona: o resultado de uma célula em um passo é calculado a partir da configuração atual, sem utilizar resultados parcialmente produzidos para outras células. Essa propriedade permite distribuir as atualizações entre threads, desde que sejam respeitadas as dependências entre as etapas. A ativação das contenções deve terminar antes da propagação, e a atualização completa deve preceder a troca das estruturas que representam as configurações atual e seguinte.

A atividade relaciona, assim, correção e desempenho em programação paralela com memória compartilhada. A distribuição das células pode reduzir o tempo de processamento, mas introduz custos de sincronização e escalonamento. Além disso, o trabalho realizado por uma célula depende de seu estado: células intactas avaliam a vizinhança, enquanto outros estados exigem transições mais simples. A quantidade de threads, a política de distribuição das iterações e o tamanho da grade são, portanto, aspectos relevantes para a avaliação experimental.

O projeto contempla duas implementações em C: uma sequencial, utilizada como referência de comparação, e outra paralela com OpenMP. Ambas devem produzir os mesmos resultados para uma mesma entrada, exceto pelo tempo de execução. Embora a inicialização empregue geração pseudoaleatória, a semente e a ordem das chamadas são fixadas, e não há decisões aleatórias durante a propagação. O modelo é o definido para a disciplina e não pretende representar um modelo físico validado de incêndios florestais.

2 Objetivos

2.1 Objetivo geral

Implementar e avaliar uma simulação determinística da propagação de incêndio, comparando uma solução sequencial em C com uma solução paralela em C/OpenMP, de modo a preservar as regras do modelo e investigar o desempenho em diferentes configurações de execução.

2.2 Objetivos específicos

- Representar a floresta em armazenamento linear e implementar a leitura, validação e inicialização dos dados conforme o enunciado.
- Implementar as transições de estado, a influência direcional do vento, os tempos de queima e a ativação de zonas de contenção.
- Desenvolver uma versão sequencial que sirva de referência para a validação da versão paralela, complementada por testes das regras do modelo.

- Distribuir a ativação e a atualização das células em uma região paralela persistente, utilizando `omp for`, SIMD, reduções e sincronização adequada.
- Verificar a equivalência das saídas entre versões e a independência dos resultados em relação à quantidade de threads e ao schedule.
- Comparar pelo menos duas políticas de escalonamento e analisar tempos, speedup e eficiência em diferentes cargas e quantidades de threads.

3 Modelagem do problema

3.1 Domínio espacial e temporal

A área é representada pelo conjunto de células

$$\Omega = \{(l, c) \mid 0 \leq l < L, 0 \leq c < C\}, \quad (1)$$

com $N = LC$ posições. O armazenamento é linear, com índice $i = lC + c$. As coordenadas podem ser recuperadas pela divisão inteira $l = i/C$ e pelo resto $c = i \bmod C$. Essa representação permite percorrer a grade por um único laço e distribuir suas iterações entre threads.

A evolução ocorre em, no máximo, P passos, numerados de 0 a $P - 1$. Uma configuração inicial é construída antes do primeiro passo. Em cada iteração, a aplicação das regras produz a configuração seguinte, que se torna a base da próxima atualização.

3.2 Parâmetros e restrições da entrada

O arquivo informa L, C, P , a quantidade de threads T , a semente e o limiar de ignição. Em seguida, especifica as componentes do vento e sua intensidade, as quantidades de focos (F) e de zonas (Z), as coordenadas dos focos e os retângulos de contenção com seus passos de ativação.

São exigidos $L, C, T, \text{LIMIAR} > 0, P, F, Z \geq 0$, componentes do vento em $\{-1, 0, 1\}$, direção diferente de $(0, 0)$ e intensidade entre 0 e 5. Os focos devem ser distintos, internos à grade e posicionados sobre células combustíveis. As zonas devem ter limites ordenados e internos à grade, com $0 \leq p_{\text{ativação}} < P$. Consequentemente, não há passo de ativação válido quando $P = 0$.

As duas aplicações recebem somente o caminho do arquivo de entrada:

```
./fire_seq entrada.txt
./fire_omp entrada.txt
```

O valor de T é obtido desse arquivo, inclusive nos experimentos que variam a quantidade de threads.

3.3 Cobertura, unidade e inicialização determinística

A cobertura e a unidade são propriedades fixas de cada célula. A inicialização percorre a matriz em ordem crescente de linhas e colunas, com uma única thread. Para cada

posição, primeiro obtém-se `rand_r(&seed) % 100` para determinar a cobertura e, imediatamente depois, `rand_r(&seed) % 101` para determinar a umidade. Preservar essa ordem é necessário para que as versões recebam a mesma configuração inicial no ambiente experimental.

Tabela 1: Coberturas e parâmetros do modelo. As proporções são as da regra de geração, não contagens exatas de cada matriz.

Código	Cobertura	Sorteio	Fator	Queima
0	Água	0–9	0	—
1	Solo exposto	10–19	0	—
2	Vegetação rasteira	20–54	8	2 passos
3	Floresta	55–99	12	4 passos

Água e solo exposto começam no estado não combustível. Vegetação rasteira e floresta começam intactas, exceto nas posições dos focos iniciais, que passam ao estado em chamas e recebem o tempo de queima de sua cobertura. A quantidade de células inicialmente combustíveis é registrada antes da evolução, incluindo os focos, para servir de denominador dos percentuais finais.

3.4 Estados e estruturas de armazenamento

O estado de uma célula pode ser não combustível (0), intacta (1), em chamas (2), queimada (3) ou contenção (4). Cobertura e estado são conceitos distintos: a cobertura identifica o material da célula; o estado indica sua situação na evolução do incêndio.

Tabela 2: Vetores utilizados para representar a floresta.

Vetor	Função
<code>cobertura</code>	Tipo de cobertura de cada posição.
<code>umidade</code>	Umidade inteira entre 0 e 100.
<code>estado_atual</code>	Configuração usada como base da atualização.
<code>proximo_estado</code>	Configuração produzida pelo passo.
<code>tempo_atual</code>	Tempo restante de queima na configuração atual.
<code>proximo_tempo</code>	Tempo restante calculado para a configuração seguinte.
<code>ativacao</code>	Primeiro passo de contenção da posição, ou -1 .

Os dois pares de vetores de estado e tempo evitam que uma célula utilize uma atualização parcial de sua vizinha. Ao fim de cada passo, os ponteiros de cada par são trocados. O armazenamento da grade é $O(LC)$; somadas as descrições dos focos e zonas, o espaço é $O(LC + F + Z)$.

3.5 Zonas de contenção

Cada zona corresponde a um retângulo de limites inclusivos e a um passo de ativação. Antes da simulação, constrói-se um mapa com o menor passo associado a cada célula nas

zonas sobrepostas; posições sem zona recebem -1 . A construção desse mapa não integra o trecho cronometrado.

No início do passo programado, apenas células intactas se tornam contenção. Os demais estados permanecem inalterados: uma zona não apaga fogo existente nem recupera células queimadas. A ativação ocorre somente no passo indicado, e as células que se tornam contenção permanecem nesse estado.

3.6 Vizinhaça e potencial de ignição

A propagação utiliza os oito vizinhos de Moore, ignorando posições fora da grade. Apenas vizinhos em chamas contribuem. Para uma célula-alvo (l, c) e um vizinho (l_v, c_v) , define-se a direção do vizinho para o alvo:

$$\Delta l = l - l_v, \quad \Delta c = c - c_v. \quad (2)$$

O peso básico b_v vale 10 se $|\Delta l| + |\Delta c| = 1$ e 7 para vizinhos diagonais. Dadas a direção fixa (w_l, w_c) e a intensidade V do vento, o alinhamento e o peso ajustado são

$$A_v = \Delta l w_l + \Delta c w_c, \quad (3)$$

$$P_v = \max(1, b_v + V A_v). \quad (4)$$

O produto escalar pode assumir valores entre -2 e 2 . Valores positivos favorecem a propagação; negativos a desfavorecem. Quando $V = 0$, o peso ajustado coincide com o básico.

Se f_i é o fator combustível e u_i a unidade da célula-alvo, seu potencial é

$$S_i = \sum_{v \text{ em chamas}} P_v, \quad I_i = \left\lfloor \frac{S_i f_i (100 - u_i)}{100} \right\rfloor. \quad (5)$$

Uma célula intacta se incendeia quando $I_i \geq \text{LIMIAR}$. Todos esses cálculos usam aritmética inteira, evitando diferenças de arredondamento na propagação entre as versões.

3.7 Transições, atualização síncrona e parada

Após a ativação das zonas, células não combustíveis, queimadas e de contenção mantêm seus estados. Células intactas permanecem intactas ou se incendeiam conforme o potencial. Células já em chamas têm seu tempo reduzido em uma unidade e tornam-se queimadas ao atingir zero.

Uma célula recém-incendiada recebe o tempo completo de sua cobertura. Ela não propaga fogo imediatamente dentro do mesmo passo: sua contribuição passa a existir quando a configuração produzida se torna a configuração atual. Essa separação preserva a atualização síncrona e permite que as células sejam processadas em ordens diferentes sem alterar o resultado.

A sequência definida para cada passo é: ativar as zonas, produzir os próximos estados e tempos, calcular as estatísticas do próximo estado, trocar os vetores e verificar a parada. A execução termina ao completar P passos ou quando não há células em chamas. Se não houver fogo após a inicialização, nenhum passo é executado, mesmo com contenções futuras.

3.8 Estatísticas e saída

As novas ignições de um passo são as transições de intacta para em chamas; os focos iniciais não integram esse total. O pico registra a maior quantidade de novas ignições e, em empate, o primeiro passo. Na ausência de novas ignições, o resultado do pico é $(-1, 0)$.

Se B_0 é a quantidade de células inicialmente combustíveis, Q a quantidade final queimada, H a quantidade final em chamas e K a quantidade final em contenção, os percentuais são

$$\text{Queimado} = 100 \frac{Q + H}{B_0}, \quad \text{Protegido} = 100 \frac{K}{B_0}. \quad (6)$$

Quando $B_0 = 0$, ambos são zero. Células ainda em chamas entram no percentual queimado porque já foram atingidas pelo incêndio.

O checksum é calculado sequencialmente após a simulação, na ordem linear da matriz. Partindo de zero, para cada célula incorpora-se primeiro o estado e depois o tempo, pela operação $h \leftarrow 31h + x$, utilizando `unsigned long long` conforme o enunciado. Ele complementa a comparação dos resultados, sem substituir a verificação das demais saídas.

A saída contém o número de passos executados, as cinco contagens finais, o total e o pico de ignições, os dois percentuais, o checksum e o tempo. Os percentuais têm duas casas decimais e o tempo tem seis; todos os campos, exceto o tempo, devem coincidir entre as versões.

4 Implementação sequencial

A versão sequencial, em `fire_seq.c`, é a referência de correção e de desempenho do trabalho. Ela segue diretamente a modelagem da Seção 3, sem otimizações que alterem a ordem das operações, de modo que cada etapa do passo corresponde a uma função com responsabilidade única.

4.1 Organização do código

Os parâmetros de entrada, os focos, as zonas e os vetores da Tabela 2 são agrupados na estrutura `Simulacao`, passada por ponteiro a todas as funções. Focos e zonas são representados pelas estruturas `Foco` e `ZonaContencao`. A Tabela 3 resume as funções e indica quais pertencem ao trecho cronometrado.

4.2 Preparação da simulação

A leitura verifica o retorno de cada `fscanf` e aplica as restrições da Seção 3.2, encerrando o programa com `EXIT_FAILURE` diante de qualquer inconsistência. A verificação de que os focos estão sobre células combustíveis só é possível após a geração da floresta e, por isso, é feita em `aplicar_focos`. Como todos os índices da simulação são do tipo `int`, a leitura também rejeita grades em que LC excede `INT_MAX` ou em que o tamanho em bytes dos vetores não cabe em `size_t`; verificação análoga é aplicada às quantidades de focos e zonas. Todas as alocações têm o retorno conferido.

Tabela 3: Funções da versão sequencial.

Função	Responsabilidade	Cronometrada
ler_entrada	Leitura e validação do arquivo e dos limites de tamanho.	Não
alocar_estruturas	Alocação dos sete vetores de tamanho LC .	Não
gerar_floresta	Sorteio da cobertura e da umidade; estado inicial.	Não
aplicar_focos	Marcação dos focos e validação da cobertura.	Não
construir_mapa_ativacao	Menor passo de ativação por célula.	Não
ativar_zonas	Conversão de intactas em contenção no passo.	Sim
potencial_ignicao	Cálculo de I_i para uma célula intacta.	Sim
calcular_proximo_estado	Transições e contagem de ignições.	Sim
trocar_matrizes	Troca dos ponteiros atual/próximo.	Sim
existe_em_chamas	Verificação da condição de parada.	Sim
calcular_checksum	Checksum na ordem linear.	Não

Na geração da floresta, a semente lida é copiada para uma variável local do tipo `unsigned int`, tipo exigido por `rand_r`, evitando a conversão de ponteiros entre `int` e `unsigned int`. As chamadas passam pela função `fire_rand_r`, que utiliza o `rand_r` nativo no Linux e, no Windows e no macOS, uma reimplementação do algoritmo da glibc. No Windows, `rand_r` não existe; no macOS, existe, mas emprega outro algoritmo e produziria uma floresta diferente para a mesma semente. Assim, a configuração inicial é a mesma nas três plataformas. A ordem das chamadas é a descrita na Seção 3.3: primeiro a cobertura e, em seguida, a umidade da mesma célula.

O mapa de ativação é inicializado com -1 e preenchido percorrendo cada retângulo; em zonas sobrepostas, prevalece o menor passo. Por fim, a quantidade de células inicialmente combustíveis B_0 é contada a partir da cobertura, antes do início da cronometragem.

4.3 Laço principal

O trecho cronometrado é delimitado por duas chamadas a `omp_get_wtime()`; o cabeçalho `omp.h` é incluído apenas para essa medição, e o programa não contém diretivas OpenMP. O laço segue a ordem definida na Seção 3.7:

```
se existe_em_chamas():
    para passo = 0 ate P-1:
        ativar_zonas(passo)
        ignicoes = calcular_proximo_estado()
        total += ignicoes
        se ignicoes > pico_qtd: pico = (passo, ignicoes)
        trocar_matrizes()
        se nao existe_em_chamas(): interromper
```

A função `ativar_zonas` percorre toda a grade e altera `estado_atual` diretamente, o

que é seguro porque ocorre antes da propagação do mesmo passo. Em `calcular_proximo_estado`, cada célula é tratada conforme seu estado: não combustíveis, queimadas e contenções são copiadas com tempo zero; células em chamas têm o tempo decrementado e passam a queimadas ao atingir zero; células intactas chamam `potencial_ignicao`. Todas as leituras são feitas em `estado_atual` e `tempo_atual`, e todas as escritas em `proximo_estado` e `proximo_tempo`, o que garante a atualização síncrona.

O potencial percorre os deslocamentos $(d_l, d_c) \in \{-1, 0, 1\}^2$, exceto $(0, 0)$, e descarta vizinhos fora da grade. Para cada vizinho em chamas, a direção até o alvo é $(-d_l, -d_c)$, a partir da qual se obtêm o peso básico, o alinhamento A_v e o peso P_v com limite inferior 1. O resultado final usa divisão inteira, equivalente ao piso para valores não negativos.

O pico de ignições é atualizado apenas quando a contagem do passo é estritamente maior que o máximo anterior, o que mantém o primeiro passo em caso de empate. Como `pico_passo` começa em -1 e `pico_qtd` em 0, o resultado $(-1, 0)$ é preservado quando não há novas ignições. A troca das configurações é feita apenas pela troca dos ponteiros, sem cópia de dados.

4.4 Finalização e custo

Após a cronometragem, são contados os estados finais, calculados os percentuais com B_0 como denominador e obtido o checksum, acumulado em `unsigned long long`, cujo estouro é aritmética modular bem definida em C. A memória é liberada ao final.

Cada passo realiza três varreduras da grade: ativação das zonas, cálculo do próximo estado e verificação de parada, esta última interrompida no primeiro foco encontrado. Como cada célula intacta examina no máximo oito vizinhos, o custo por passo é $O(LC)$ e o custo total do trecho cronometrado é $O(PLC)$. O trabalho por célula, porém, não é uniforme: células intactas são mais caras que as demais, o que é relevante para a escolha do escalonamento na versão paralela.

5 Implementação paralela

A versão paralela, em `fire_omp.c`, parte do código sequencial e mantém idênticas a leitura, a validação, a alocação, a geração da floresta, a aplicação dos focos, a construção do mapa de ativação e o cálculo dos resultados finais. Todas essas etapas estão fora do trecho cronometrado, e a geração da floresta deve, por exigência do enunciado, ser feita por uma única thread. A paralelização concentra-se, portanto, no laço de passos, encapsulado na função `simular`.

5.1 Estratégia de paralelização

Como a atualização é síncrona, o próximo estado de cada célula depende apenas de `estado_atual`, `tempo_atual` e dos dados fixos, e cada iteração escreve somente na própria posição de `proximo_estado` e `proximo_tempo`. Não há, assim, dependência entre iterações de um mesmo passo, e o laço sobre as células pode ser dividido entre threads por decomposição de domínio sobre o índice linear. Entre passos, por outro lado, a dependência é total: o passo $p + 1$ só pode começar depois que toda a configuração do passo p estiver pronta.

Em vez de abrir uma região paralela a cada passo, o programa utiliza uma única região `parallel` persistente, que envolve todo o laço `while`. As threads são criadas uma vez e percorrem juntas os passos, o que evita o custo repetido de criação e sincronização da equipe ao longo de até P passos. Dentro dela, cada passo é organizado da seguinte forma:

```
#pragma omp parallel num_threads(T)
while (continua):
    #pragma omp for schedule(static)           // ativar_zonas
    #pragma omp for schedule(guided, 1024) \
        reduction(+:ignicoes) reduction(||:tem_chamas)
    #pragma omp single                         // estatisticas, troca, parada
```

A ativação das zonas é feita em `ativar_zonas` por um `omp for` órfão, isto é, uma diretiva de compartilhamento de trabalho fora do escopo léxico da região paralela, que se associa à equipe em execução. Cada iteração lê `ativacao[i]`, que é fixo, e escreve apenas em `estado_atual[i]`, sem conflito entre threads.

A atualização das células foi reorganizada na função `atualizar_celula`, que processa uma única posição e não contém diretivas OpenMP; o laço que a chama é o que recebe o `omp for`. Além do indicador de nova ignição, a função informa se a célula estará em chammas no próximo estado. Esses valores são combinados por duas reduções: soma para as novas ignições e disjunção lógica para a existência de fogo. Com isso, a condição de parada passa a ser obtida no próprio laço de atualização, e a varredura adicional de `existe_em_chamas` a cada passo, presente na versão sequencial, deixa de ser necessária. A função continua sendo usada apenas na verificação inicial, antes da região paralela; se não houver fogo ou se $P = 0$, a região nem chega a ser aberta.

Em `potencial_ignicao`, os oito deslocamentos de Moore são armazenados em vetores constantes, e o laço sobre eles foi reescrito sem desvios (`continue`), para permitir a vetorização com `omp simd reduction(+:S)`. Um vizinho fora da grade tem o índice substituído pelo da própria célula, sempre válido, e sua contribuição é anulada pelo indicador `dentro`; o mesmo ocorre com vizinhos que não estão em chammas. A soma S_i resultante é a mesma da versão sequencial.

5.2 Compartilhamento e sincronização

A região paralela usa `default(none)`, o que obriga a declarar explicitamente o escopo de cada variável. São compartilhados o ponteiro para a estrutura `Simulacao`, o tamanho n e as variáveis de controle do laço: `passo`, `continua`, o total e o pico de ignições e os dois acumuladores das reduções. O índice dos laços `omp for` e as variáveis locais de `atualizar_celula` e `potencial_ignicao` são privados de cada thread. Os vetores da simulação são compartilhados, mas nenhuma posição é escrita por mais de uma thread no mesmo laço.

A corretude depende das barreiras implícitas ao final de cada construção de compartilhamento de trabalho, que reproduzem a ordem da Seção 3.7:

- a barreira ao final da ativação garante que todas as contenções do passo estejam aplicadas antes que qualquer thread avalie um vizinho;
- a barreira ao final da atualização garante que todos os próximos estados estejam escritos e que as reduções estejam combinadas antes do bloco `single`;

- a barreira ao final do `single` garante que a troca de ponteiros e os novos valores de `passo` e `continua` sejam vistos por todas as threads antes da próxima iteração do `while`.

As operações de custo constante do passo, como a atualização do total e do pico, a troca dos vetores, o incremento de `passo` e o cálculo de `continua`, são feitas por uma única thread no bloco `single`. O mesmo bloco zera os acumuladores das reduções para o passo seguinte, o que dispensa um segundo `single` e uma barreira adicional por passo. Não há uso de `critical` nem de `atomic`: toda a combinação de resultados parciais é feita pelas reduções.

O número de threads é definido por `num_threads(T)`, com o valor lido do arquivo de entrada, e `omp_set_dynamic(0)` é chamado antes da região para impedir que o ambiente reduza o tamanho da equipe. Como as reduções operam sobre inteiros, cuja soma é associativa, os resultados não dependem da ordem em que as threads combinam seus valores parciais. As saídas são, portanto, as mesmas para qualquer quantidade de threads e qualquer política de escalonamento.

5.3 Escalonamento

Os dois laços paralelos têm perfis de custo distintos e, por isso, utilizam políticas diferentes. A ativação das zonas executa trabalho praticamente uniforme por iteração: uma comparação e, eventualmente, uma escrita. Para ela, usa-se `schedule(static)`, que atribui a cada thread um bloco contíguo e fixo de índices, com sobrecarga mínima e boa localidade de memória. A política é declarada explicitamente porque, sem a cláusula, o escalonamento adotado depende da implementação.

Na atualização das células, o custo por iteração é irregular. Células intactas calculam o potencial sobre oito vizinhos, enquanto as demais apenas copiam ou decrementam valores. A distribuição desse custo muda ao longo da simulação, à medida que a frente de fogo avança e as regiões queimadas e contidas crescem. Uma divisão estática em blocos iguais pode atribuir a uma thread uma faixa com muito mais células intactas que a de outra. Por isso, o laço usa `schedule(guided, 1024)`: os blocos começam grandes e diminuem conforme o trabalho restante, equilibrando a carga no final do laço. O tamanho mínimo de 1024 iterações limita a quantidade de requisições ao escalonador e preserva a localidade, evitando a sobrecarga de políticas como `dynamic` com blocos unitários.

A comparação entre esta política e outras alternativas, exigida pelo enunciado, é apresentada na Seção 8.

6 Validação da correção

Para cada carga, o checksum e a quantidade de passos executados coincidiram em todas as 90 execuções (versão sequencial e versão OpenMP com T de 2 a 8, dez repetições cada), confirmando que o resultado independe da quantidade de threads, conforme exigido pelo enunciado. Nas três cargas a simulação consumiu todos os P passos configurados, isto é, o incêndio não se extinguiu antes do limite definido na Tabela de cargas. As entradas com zero passos, sem focos e com zonas de contenção sobrepostas foram contempladas na validação da entrada (Seção 3.2) e na lógica de `construir_mapa_ativacao`, mas ainda

Tabela 4: Validação da versão final (macOS).

Caso	Threads	Schedule	Resultado
Carga pequena	pe- 1–8	guided(1024)	Checksum e passos idênticos entre <code>fire_seq</code> e <code>fire_omp</code> nas 90 execuções (checksum 17073017104646724864; 80 passos)
Carga média	1–8	guided(1024)	Checksum e passos idênticos nas 90 execuções (checksum 17187382406529706254; 100 passos)
Carga grande	1–8	guided(1024)	Checksum e passos idênticos nas 90 execuções (checksum 14825089465781038537; 100 passos)

não possuem uma execução automatizada dedicada nos scripts de teste; essa cobertura será complementada nos experimentos do cluster Linux.

7 Ambiente e metodologia experimental

7.1 Ambiente das duas versões

Tabela 5: Ambiente experimental — macOS.

Item	Configuração
CPU e núcleos	Apple M4 (4 núcleos de desempenho + 6 de eficiência; 10 núcleos no total, sem SMT/hyperthreading)
Memória e sistema	16 GiB de RAM; macOS 26.5.2 (build 25F84); arquitetura arm64 (Apple Silicon)
Compilador	Apple clang 17.0.0 (clang-1700.6.4.2), target=arm64-apple-darwin25.5.0, com libomp do Homebrew (/opt/homebrew/opt/libomp)
Compilação sequencial	Makefile.macos, alvo <code>fire_seq</code> : <code>clang -O2 -Wall -Wextra -std=c11 -Xpreprocessor -fopenmp -I<libomp>/include -L<libomp>/lib -lomp fire_seq.c -o fire_seq</code>
Compilação paralela	mesmos parâmetros de compilação, alvo <code>fire_omp</code> (<code>fire_omp.c</code>)
Execução OpenMP	T lido do arquivo de entrada e fixado com <code>num_threads(T)</code> e <code>omp_set_dynamic(0)</code> ; a política de escalonamento do laço de atualização (<code>guided(1024)</code> nesta rodada) é fixada em tempo de compilação na diretiva <code>omp for</code>
Versão dos fontes	commit <code>fe355af</code> (branch <code>main</code>)

Observação sobre o ambiente. Todos os resultados apresentados na Seção 8 e nas seções seguintes desta versão do relatório foram obtidos exclusivamente no ambiente macOS descrito acima, com o script `scripts/run_macos.sh`. Os experimentos equivalentes no cluster Linux da disciplina (usando `scripts/run_linux.sh` e `Makefile`) ainda serão executados; quando disponíveis, serão incorporados a uma versão revisada deste relatório, junto com a comparação de uma segunda política de escalonamento exigida pelo enunciado.

7.2 Cargas e protocolo de execução

Tabela 6: Cargas fornecidas. T indica o valor original, que pode variar nos experimentos.

Carga	L	C	P	T	F	Z
Pequena	400	500	80	4	3	2
Média	1200	1500	100	8	6	4
Grande	2500	2500	100	8	10	6

Cada configuração foi executada pelo script `scripts/run_macos.sh`: a versão sequencial é executada uma vez por repetição (sem variar threads) e a versão paralela é executada para $T \in \{1, \dots, 8\}$, com o 4º campo da primeira linha da entrada sobrescrito temporariamente a cada valor de T testado; os demais parâmetros da Tabela acima são mantidos. Foram realizadas 10 repetições por combinação de carga e T , totalizando $3 \times (10 + 8 \times 10) = 270$ execuções nesta rodada. Nesta versão do relatório, apenas a política `guided(1024)` foi medida no macOS; a comparação com uma segunda política de escalonamento, exigida pelo enunciado, será obtida junto com os experimentos no cluster Linux.

7.3 Delimitação do trecho cronometrado

O recorte especificado inclui a ativação das zonas, a propagação, a atualização dos estados e tempos, as estatísticas, a troca das matrizes e a verificação da parada. Exclui a leitura, a validação, a alocação, a geração da floresta, a definição dos focos, a construção do mapa, o checksum, os percentuais e a impressão. A medição deve utilizar `omp_get_wtime()`.

7.4 Métricas

Para a mesma carga, o speedup e a eficiência percentual são definidos por

$$S(T) = \frac{t_{\text{seq}}}{t_{\text{omp}}(T)}, \quad E(T) = \frac{S(T)}{T} \times 100\%. \quad (7)$$

A referência é o programa sequencial. A versão OpenMP com uma thread é uma configuração adicional e não substitui essa referência.

8 Resultados

8.1 MacOs

Os resultados desta seção foram obtidos no ambiente macOS descrito na Seção 7, com a política `guided(1024)`. Cada valor de t_{seq} e t_{omp} é a média aritmética de 10 repetições; o script de execução também registrou o desvio entre repetições (não incluído na tabela), pequeno para as cargas média e grande e um pouco maior, em termos relativos, para a carga pequena, cujo tempo de execução é da ordem de dezenas de milissegundos. Os experimentos no cluster Linux da disciplina, incluindo a comparação com uma segunda

política de escalonamento, serão incorporados a uma versão revisada desta tabela e dos gráficos.

Tabela 7: Tempos e métricas de desempenho (macOS, Apple M4, política `guided(1024)`).

Carga	Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
Pequena	guided(1024)	2	0.083092	0.052361	1.59	79.35
	guided(1024)	3	0.083092	0.035459	2.34	78.11
	guided(1024)	4	0.083092	0.032095	2.59	64.72
	guided(1024)	5	0.083092	0.029900	2.78	55.58
	guided(1024)	6	0.083092	0.028959	2.87	47.82
	guided(1024)	7	0.083092	0.027257	3.05	43.55
	guided(1024)	8	0.083092	0.033282	2.50	31.21
Média	guided(1024)	2	0.942207	0.588427	1.60	80.06
	guided(1024)	3	0.942207	0.456809	2.06	68.75
	guided(1024)	4	0.942207	0.376859	2.50	62.50
	guided(1024)	5	0.942207	0.346127	2.72	54.44
	guided(1024)	6	0.942207	0.328233	2.87	47.84
	guided(1024)	7	0.942207	0.300556	3.14	44.78
	guided(1024)	8	0.942207	0.278883	3.38	42.23
Grande	guided(1024)	2	3.302497	2.267915	1.46	72.81
	guided(1024)	3	3.302497	1.727732	1.91	63.72
	guided(1024)	4	3.302497	1.422973	2.32	58.02
	guided(1024)	5	3.302497	1.341581	2.46	49.23
	guided(1024)	6	3.302497	1.258350	2.62	43.74
	guided(1024)	7	3.302497	1.252327	2.64	37.67
	guided(1024)	8	3.302497	1.181848	2.79	34.93

9 Análise e discussão

Todas as observações abaixo se referem exclusivamente ao ambiente macOS descrito na Seção 7 e à política `guided(1024)`; conclusões sobre outros escalonamentos e sobre o comportamento no cluster Linux ficam para a versão revisada deste relatório.

Ganho de threads limitado pelo número de núcleos de desempenho. O Apple M4 utilizado possui apenas 4 núcleos de desempenho e 6 de eficiência, sem SMT. Para as três cargas, o speedup cresce de forma quase linear até $T = 3$ –4 (por exemplo, $S(4) = 2.50$ na carga média) e passa a crescer bem mais devagar a partir daí: entre $T = 4$ e $T = 8$ a carga média ganha apenas 0.88 de speedup adicional, contra 1.66 entre $T = 1$ e $T = 4$. Esse comportamento é compatível com threads adicionais sendo atendidas por núcleos de eficiência, mais lentos que os de desempenho, e explica por que $E(T)$ cai de forma monótona em todas as cargas: de 83–84% em $T = 1$ para 31–43% em $T = 8$.

A carga grande escala pior do que a carga média. Em $T = 8$, a carga média atinge $S(8) = 3.38$ e $E(8) = 42.23\%$, enquanto a carga grande atinge apenas $S(8) = 2.79$ e $E(8) = 34.93\%$, apesar de ter mais trabalho por passo (2500×2500 contra 1200×1500 células). Esse resultado é o oposto do que a lei de Amdahl prevista isoladamente sugeriria (cargas maiores tendem a diluir melhor a fração serial fixa por passo). Uma explicação

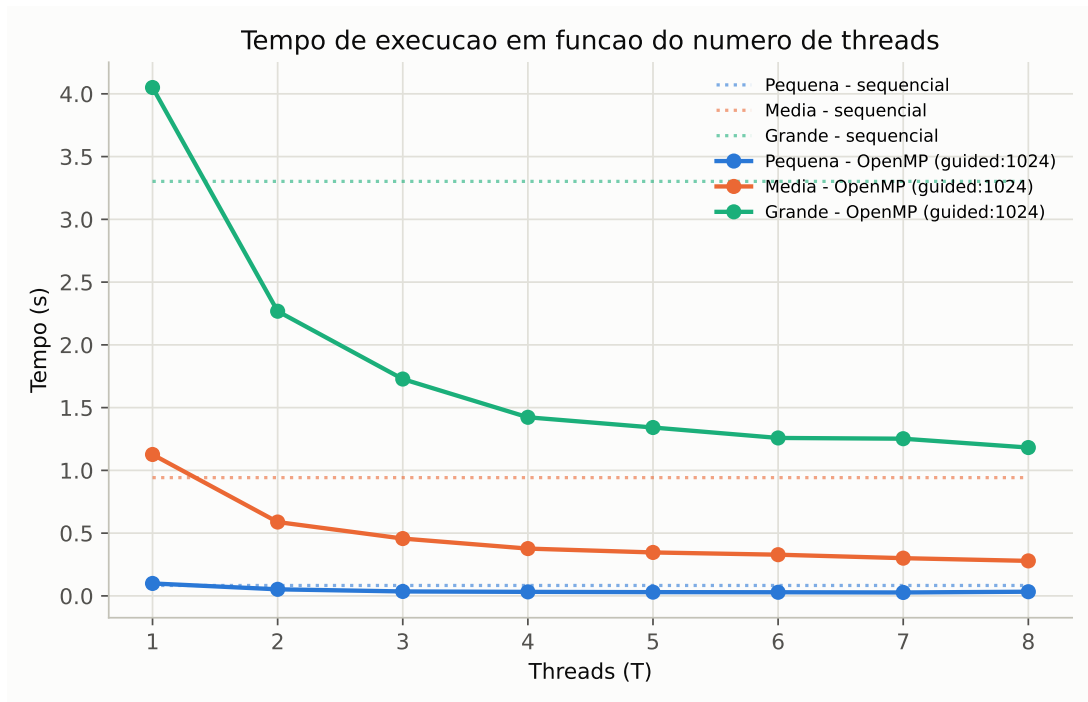


Figura 1: Tempo em função do número de threads. Identificar cargas e schedules.

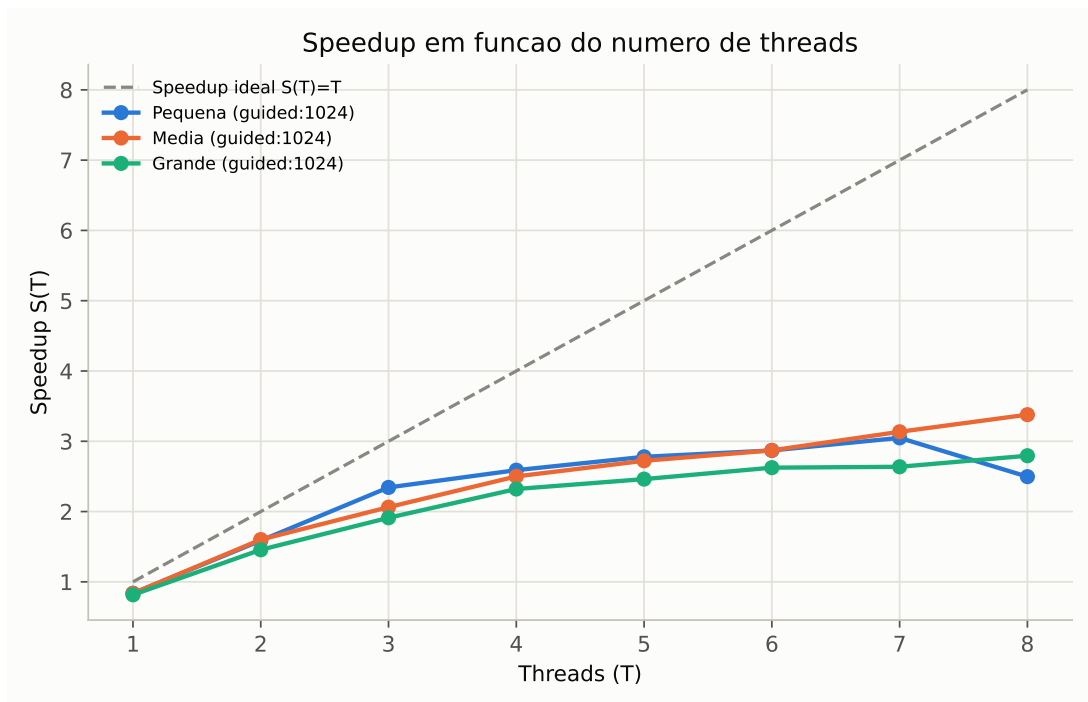


Figura 2: Speedup em função do número de threads, com referência ideal $S(T) = T$.

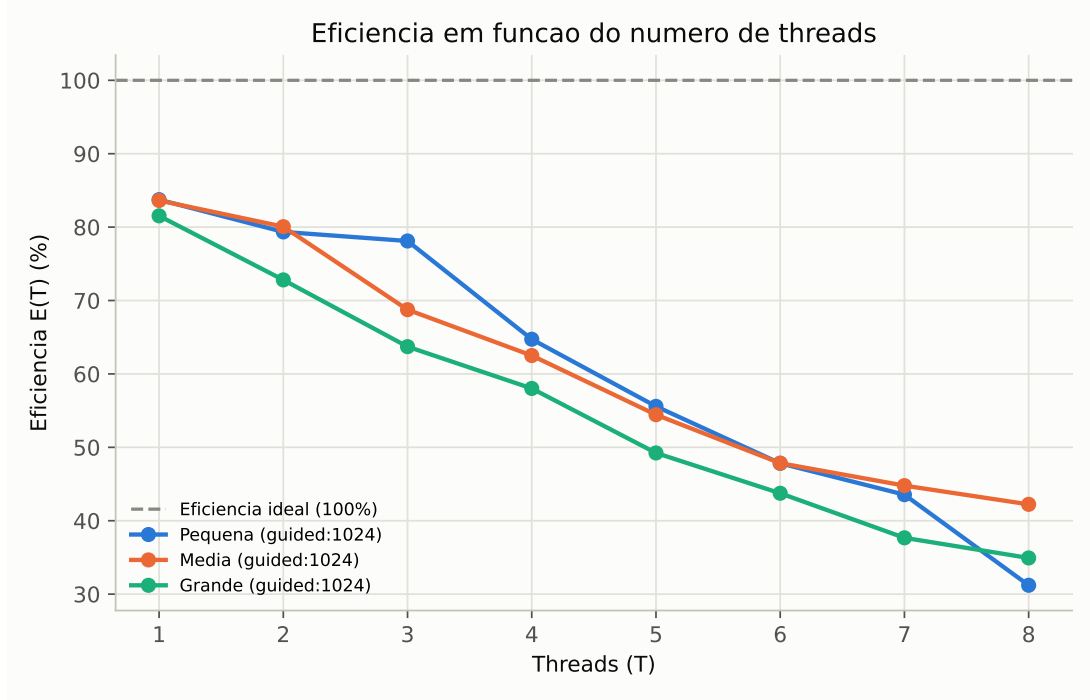


Figura 3: Eficiência percentual em função do número de threads.

provável é a pressão sobre a hierarquia de memória: a grade da carga grande ($\approx 6,25$ milhões de células, com sete vetores de tamanho LC) não cabe nos caches compartilhados pelos núcleos de desempenho, de modo que, ao aumentar o número de threads ativas, a carga passa a ser limitada pela largura de banda de memória, e não apenas pelo paralelismo de cálculo. Essa hipótese deve ser reexaminada no cluster Linux, cuja hierarquia de cache e memória é distinta.

Queda de speedup na carga pequena em $T = 8$. A carga pequena tem o pior comportamento relativo em threads altas: o speedup sobe até $S(7) = 3.05$ e cai para $S(8) = 2.50$, o único caso, entre as três cargas, em que aumentar T piora o tempo de execução. Como o trecho cronometrado da carga pequena dura poucas dezenas de milissegundos e tem apenas 200 000 células distribuídas por passo, o custo fixo de cada barreira implícita e de cada região de trabalho compartilhado (Seção 3.7) passa a ser proporcionalmente maior à medida que T cresce e que a fatia de trabalho por thread diminui; com 8 threads, cada uma recebe em média 25 000 células por passo, valor próximo do tamanho mínimo de bloco do `guided(1024)`, o que reduz a eficácia do balanceamento de carga e evidencia o custo de sincronização por passo. Esse efeito é uma evidência de que, para cargas pequenas, threads demais podem ser contraproducentes com essa política de escalonamento — um ponto a ser confrontado com a segunda política de escalonamento ainda pendente.

Consistência dos resultados. Como reportado na Seção 8 anterior (validação), o checksum e a quantidade de passos executados foram idênticos entre `fire_seq` e `fire_omp` para todo T de 1 a 8, nas três cargas, confirmando que a paralelização não altera o resultado da simulação — apenas o seu tempo de execução, exatamente como esperado da modelagem com atualização síncrona e reduções (Seção 3.7).

10 Conclusão

As implementações sequencial e paralela atendem aos requisitos do enunciado: mesma saída determinística para uma dada entrada, independentemente da quantidade de threads, com a versão paralela construída sobre uma região `parallel` persistente, `omp for` com `default(none)`, reduções para os contadores e SIMD no cálculo do potencial de ignição. No ambiente macOS avaliado (Apple M4, 10 núcleos, política `guided(1024)`), a versão OpenMP produziu speedups entre $2.50\times$ e $3.38\times$ em relação à versão sequencial, com eficiência decrescente à medida que a quantidade de threads ultrapassa o número de núcleos de desempenho da máquina. A carga grande escalou pior do que a média, indicando possível limitação de banda de memória, e a carga pequena chegou a piorar ao usar 8 threads, evidenciando que o custo de sincronização por passo pode superar o ganho de paralelismo quando a carga de trabalho é pequena.

Esses resultados são específicos do ambiente macOS descrito na Seção 7 e de uma única política de escalonamento. Os próximos passos deste trabalho são: (i) executar o mesmo protocolo experimental no cluster Linux da disciplina, com `scripts/run_linux.sh` e o `Makefile` padrão; (ii) medir pelo menos uma segunda política de escalonamento (por exemplo, `static` ou `dynamic`) para o laço de atualização, conforme exigido pelo enunciado; e (iii) completar a validação de correção com os casos de zero passos/sem focos e de zonas de contenção sobrepostas. Espera-se que o cluster Linux, com mais núcleos e uma hierarquia de memória diferente da do M4, permita avaliar se a queda de eficiência observada aqui é uma limitação da arquitetura específica ou do algoritmo de paralelização em si.

Referências

- [1] Universidade de São Paulo, ICMC/SSC. *SSC0903 — Primeiro Trabalho Prático: simulação paralela da propagação direcional de incêndio com zonas de contenção*. Arquivo `Trab01-Fire-OMP-v1.pdf`, setembro de 2026.